Extraction of Validating Shapes from very large Knowledge Graphs [Scalable Data Science]

Kashif Rabbani
Aalborg University, Denmark
kashifrabbani@cs.aau.dk

Matteo Lissandrini
Aalborg University, Denmark
matteo@cs.aau.dk

Katja Hose
Aalborg University, Denmark
khose@cs.aau.dk

ABSTRACT

Knowledge Graphs (KGs) represent heterogeneous domain knowledge on the Web and within organizations. There exist shapes constraint languages to define *validating shapes* to ensure the quality of the data in KGs. Existing techniques to extract validating shapes often fail to extract complete shapes, are not scalable, and are prone to produce spurious shapes. To address these shortcomings, we propose the Quality Shapes Extraction (QSE) approach to extract validating shapes in very large graphs, for which we devise both an exact and an approximate solution. QSE provides information about the reliability of shape constraints by computing their confidence and support within a KG and in doing so allows to identify shapes that are most informative and less likely to be affected by incomplete or incorrect data. To the best of our knowledge, QSE is the first approach to extract a complete set of validating shapes from WikiData. Moreover, QSE provides a 12x reduction in extraction time compared to existing approaches, while managing to filter out up to 93% of the invalid and spurious shapes, resulting in a reduction of up to 2 orders of magnitude in the number of constraints presented to the user, e.g., from 11,916 to 809 on DBpedia.

PVLDB Reference Format:
Kashif Rabbani, Matteo Lissandrini, and Katja Hose. Extraction of Validating Shapes from very large Knowledge Graphs [Scalable Data Science]. PVLDB, 16(1): XX-XX, 2023.
doi:X.X/XX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/dkw-aau/qse.

1 INTRODUCTION

Knowledge Graphs (KGs) are in widespread use both within companies [40, 41] and on the Web [44, 47]. They represent entities and their relationships in various domains [27, 41] as collections of $\langle subject, relation, object \rangle$ triples using the Resource Description Framework (RDF) [8]. KGs are key to various data-centric AI tasks, such as search engines, question answering, and smart assistants [16, 27]. Nonetheless, as more and more data is accrued within KGs, practical applications impose further demands, especially in terms of quality assessment and validation [33, 37, 50]. Therefore, Shapes constraint languages, e.g., SHACL [21], and ShEx [32], have

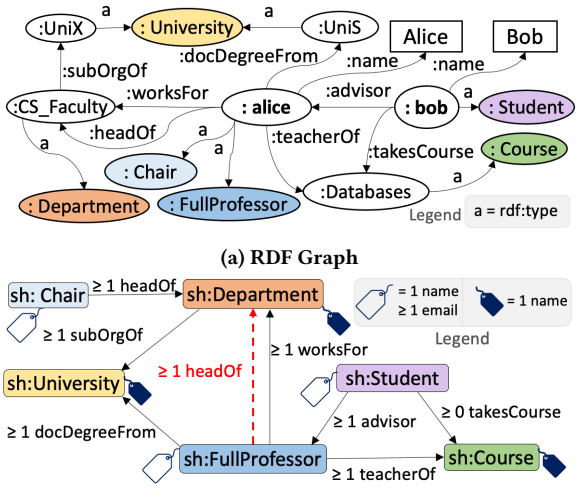

Figure 1: An example RDF Graph and Validating Shapes

been proposed to validate KGs by enforcing constraints represented in the form of *validating shapes*. For instance, we can express that an entity of type Student requires a name, a registration number, and should be enrolled to some courses; and that these attributes should be of type string, date, and Course, respectively – see Figure 1a for an example KG and Figure 1b for a simplified depiction of its corresponding set of validating SHACL shapes. Thus, data scientists can ensure the quality of a given KG or can identify errors within it by validating its content against a set of validating shapes.

Often, validating shapes are manually specified by domain experts. Yet, when trying to specify validating shapes for already-existing large-scale KGs, data scientists are in need of tools that can speedup this process [37]. Thus, various tools have been proposed to automatically [7, 10, 18, 24] or semi-automatically [3, 30, 35] produce a set of validating shapes for a target KG. Unfortunately, these methods suffer from 3 important limitations: (1) they are not able to produce complete shapes, e.g., they can identify that a student should have a property of type takesCourse but they do not extract the fact that the object should be of type Course; (2) the shapes they produce are easily affected by errors and inconsistencies in the KG, e.g., if some departments, by mistake, are attached the property hasAdvisor, a corresponding *spurious shape* is extracted; and (3) they do not scale to large KGs, i.e., they cannot process at all the full English WikiData and they take days to process a subset of it. Therefore, in this work, we aim at supporting data scientists by presenting the first techniques for *efficient extraction of validating shapes from very large existing KGs that also ensures robustness against the effects of spuriousness.*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 1 ISSN 2150-8097.
doi:X.X/XX.XX

We highlight how the effect of spuriousness is particularly vexing for automatic shape extraction methods, and at the same time, the existence of this issue is one of the major motivations behind the need for extracting validating shapes, thus it cannot be ignored. For instance, in DBpedia [2], some of the entities representing musical bands are wrongly assigned to the class dbo:City. As a consequence, when shapes are extracted from its instance data using existing approaches, the resulting node shape for dbo:City specifies that cities are allowed optional properties like dbo:genre and dbo:formerBandMember. Since existing KGs are very rich in terms of node and edge types, and due to the effect of *spuriousness*, existing approaches generate tens of thousands of shapes (our experiments show that more than 82K node shapes and 2 million property shapes are generated by a standard extraction process for WikiData [47]). Thus it becomes unmanageable for domain experts to manually curate the generated shapes to identify the valid ones. The only existing approach that attempts to tackle this issue is SheXer [10], which supports the filtering of shapes based on a "trustworthiness" score. Unfortunately, this score does not directly translate into how frequently a shape is satisfied in a dataset, so it is still prone to generate spurious shapes and it is also hard to tune. Furthermore, SheXer is not designed to handle very large KGs. Our experiments showed that it exceeded the available memory (256 GB) while trying to process the English subset of WikiData.

Therefore, to tackle the issue of *spuriousness*, we study and formalize the problem of *support-based shapes extraction* and propose the Quality Shapes Extraction (QSE) approach as a solution to this problem. To tackle the issue of *scalability*, we devise two efficient algorithms, QSE-Exact and QSE-Approximate. These efficient algorithms implement the processes of *constraints extraction* and *computation of support and confidence* in order to enable the *construction of SHACL shapes* with *quality guarantees* from very large KGs. Support and confidence represent the number and ratio of the nodes and properties in the graph conforming to a given shape. Users can specify a minimum threshold for support and confidence for the constraints that are going to be extracted, similarly to analogous scores for frequent itemset mining [4, 17]. *Hence, QSE can filter out shapes affected by spurious or erroneous data based on robust and easily understandable measures.* Moreover, our efficient approximation algorithm (QSE-Approximate) enables shape extraction on a commodity machine by sampling the KG entities via a dynamic multi-tiered reservoir sampling technique.

We perform a thorough experimental evaluation using both synthetic (LUBM [15]) and real (DBpedia [2], YAGO-4 [44], WikiData [47]) KGs demonstrating the benefits of our approach. The results show that our QSE-Exact approach can extract shapes from the entire WikiData's 2015 dump in 16 minutes and from the 2021's dump in less than 3 hours. Similarly, our QSE-Approximate approach is able to extract shapes from WikiData's 2021 dump in 90 minutes on a 32GB machine while still achieving 100% precision and 95% recall in the set of shapes produced.

2 RDF SHAPES AND THE QSE PROBLEM

In this section, we first introduce the KG data model and the concepts of validating shapes, their support, and confidence, then we define the problem of Quality Shapes Extraction which is the main focus of our paper.

2.1 Preliminaries

The standard model for encoding knowledge graphs is the Resource Description Framework (RDF [8]). It is a W3C standard format to publish and exchange data over the Web as a finite set of $\langle s, p, o\rangle$ triples stating that a subject s is in a relationship with an object o through predicate p. Therefore, we define an RDF graph as follows:

Definition 2.1 (RDF graph). Given pairwise disjoint sets of IRIs $\mathcal{I}$, blank nodes $\mathcal{B}$, and literals $\mathcal{L}$, an RDF Graph $\mathcal{G}{:}\langle N, E\rangle$ is a graph with a finite set of nodes $N{\subset}(\mathcal{I}{\cup}\mathcal{B}{\cup}\mathcal{L})$ and a finite set of edges $E{\subset}\{\langle s, p, o\rangle{\in}(\mathcal{I}{\cup}\mathcal{B}) \times \mathcal{I} \times (\mathcal{I}{\cup}\mathcal{B}{\cup}\mathcal{L})\}$.

Moreover, we distinguish two special subsets of the IRIs $\mathcal{I}$: predicates $\mathcal{P}$ and classes $\mathcal{C}$. The set of predicates $\mathcal{P}{\subset}\mathcal{I}$ is the subset of IRIs that appear in the predicate position p in any $\langle s, p, o\rangle{\in}\mathcal{G}$. Among predicates $\mathcal{P}$, we identify the type predicate $\mathbf{a}{\in}\mathcal{P}$, which corresponds to IRI rdf:type [49] or wdt:P31 WikiData [47], as the predicate that connects all entities that are instances of a class to the node representing the class itself, i.e., their type. Thus, all the IRIs that are classes in $\mathcal{G}$ form the subset $\mathcal{C}{:}\{c{\in}\mathcal{I}\,|\,\exists s{\in}\mathcal{I}$ s.t. $\langle s, \mathbf{a}, c\rangle{\in}\mathcal{G}\}$.

Given a KG $\mathcal{G}$, a set of *validating shapes* represents integrity constraints in the form of a shape schema $\mathcal{S}$ over $\mathcal{G}$. Since the shape schema describes shapes associated with node types and their connections to other attributes and node types, we can also visualize the shape schema $\mathcal{S}$ as a particular type of graph (see Figures 1a and 1b). Therefore, in the following, we refer to two concepts: the *data graph* $\mathcal{G}$ and the *shape graph* derived from $\mathcal{S}$. The *data graph* is the RDF graph $\mathcal{G}$ to be validated, while the *shape graph* consists of constraints in the form of the shape schema $\mathcal{S}$ against which entities of the data graph are validated. These constraints are defined using node and property shapes. In the following, we define the syntax of $\mathcal{S}$ according to SHACL *core constraint components* [48] by adopting the syntax introduced by Ognjen et al. [39]. Nevertheless, validating shapes can also be expressed in ShEx [33] and our approach can trivially be extended to support that syntax as well.

Definition 2.2 (Shape Schema). A SHACL shape schema consists of a set of node shapes $\mathcal{S}$, with $\langle s, \tau_s, \Phi_s\rangle{\in}\mathcal{S}$, where s is the shape *name*, $\tau_s{\in}\mathcal{C}$ is the *target class*, and Φ_s is a set of property shapes of the form $\phi_s{:}\langle\tau_\mathsf{p}, \mathsf{T}_\mathsf{p}, \mathsf{C}_\mathsf{p}\rangle$, where $\tau_\mathsf{p}{\in}\mathcal{P}$ is called the target property, $\mathsf{T}_\mathsf{p}{\subset}\mathcal{I}$ contains either an IRI defining a *literal type*, e.g., xsd:string, or a set of IRIs – called *class type constraint*, and C_p is a pair $(n, m) \in \mathbb{N} \times (\mathbb{N}{\cup}\{\infty\})$. $n{\leq}m$ – called min and max *cardinality constraints*.

Therefore, given a node shape $s{\in}\mathcal{S}$ for the *target class* $\tau_s{\in}\mathcal{C}$, Φ_s defines which properties each instance of τ_s can or should be associated with. For instance, the shape $\langle$sh:Student, :Student, $\{\phi_{s_1}, \phi_{s_2}\}\rangle$ from Figure 1b, contains a node shape for target class :Student and enforces two property shapes ϕ_1 and ϕ_2 The property shape ϕ_1 has a target property τ_p= :name, a literal type constraint T_p= xsd:string, and the cardinality constraints C_p=(1, 1). Similarly, the property shape ϕ_2 has a target property τ_p=:takesCourse, a class type constraint T_p= :Course, and the cardinality constraint $\mathsf{C}_\mathsf{p} = (1, \infty)$.

When validating a graph $\mathcal{G}$ against a shape schema $\mathcal{S}$ having a node shape $\langle s, \tau_s, \Phi_s\rangle{\in}\mathcal{S}$, we verify that each entity $e{\in}\mathcal{G}$ that is instance of τ_s satisfies all the constraints Φ_s. Note that we use the term entity and node interchangeably throughout the paper. Thus, we define the semantics of $\mathcal{S}$ as follows:

Definition 2.3 (Validating Shape Semantics). Given a node shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$, a graph $\mathcal{G}$, and an entity e s.t. $\langle e, \mathbf{a}, \tau_s \rangle \in \mathcal{G}$, we have that s validates e, and we write $e \models_{\mathcal{G}} \phi$, if for every property shape $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$ the following conditions hold:

- If $\mathsf{T_p}$ is a literal type constraint, then for every triple $(e, \tau_p, l) \in \mathcal{G}$, l is a literal of type $\mathsf{T_p}$.
- If $\mathsf{T_p}$ is a set of class type constraints $\mathsf{T_p}=\{t_1, t_2, ...t_n\}$, then for every triple $(e, \tau_p, o) \in \mathcal{G}$, it holds that $\forall t \in \mathsf{T_p}$, o is an instance of t (or of a subclass of t) and if $\exists S_t \in \mathcal{S}$, $o \models_{\mathcal{G}} S_t$.
- $n \leq |\{(s, p, o) \in \mathcal{G} : s = e \wedge p = \tau_p\}| \leq m$, where $\mathsf{C_p}=(n, m)$.

Here we study the case where the knowledge graph $\mathcal{G}$ is given, and we want to extract the set of validating shapes $\mathcal{S}$ that validates every class in C from $\mathcal{G}$. This is the *shapes extraction* problem. In this case, existing automatic approaches [37] assume the graph to be correct, then iterate over all entities in it, and extract for each entity e all necessary shapes that validate e. The union of all such shapes is assumed to be the final schema $\mathcal{S}$. This is useful when we want to validate new data that will be added *in the future* to the KG so that it will conform to the data already in the graph. Unfortunately, this approach will produce spurious shapes. For instance, in Figure 1, since :alice has both type Full Professor and Chair, when parsing the triple (:alice, :headOf, :CS_Faculty), the property shape headOf (the red dotted arrow in Figure 1b) is assigned to both node shapes, instead of assigning it to the Chair node shape only.

2.2 Shapes Support and Confidence

To contrast the effect of spuriousness, we want to exploit statistics on how often properties are applied to entities of a given type. Therefore, we introduce the notion of *support* and *confidence* for shape constraints to study the reliability of extracted shapes. These concepts are inspired by the well-known theory developed for the task of frequent patterns mining [17] and the concept of MNI support for graph patterns [5]. The MNI support of a graph pattern is the minimum cardinality of the *set of all nodes of $\mathcal{G}$ that are mapped to a specific pattern node by some isomorphism* across all the nodes of the pattern. In our approach, a property shape corresponds to a node- and edge-labeled graph pattern. Thus, given the shape $s{:}\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ its support is the number of entities that are of type τ_s, while the support of a property shape $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$ is the cardinality of entities conforming to it.

Definition 2.4 (Support of ϕ_s). Given a shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ with shape constraint $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$, the support of ϕ_s is defined as the number of entities e satisfying ϕ_s, denoted as $e \models \phi_s$, hence:

$$\text{supp}(\phi_s) = |\{e \in \mathcal{I} \mid e \models \phi_s\}| \tag{1}$$

Finally, the confidence of a constraint ϕ_s measures the ratio between how many entities conform to ϕ_s and the total number of entities that are instances of the target class of the shape s.

Definition 2.5 (Confidence of ϕ_s). Given a shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ having shape constraint $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$, the confidence of ϕ_s is defined as the proportion of entities for which $e \models \phi_s$ among the entities that are instances of the target class τ_s of $s \in \mathcal{S}$, hence:

$$\text{conf}(\phi_s) = \frac{\text{supp}(\phi_s)}{|\{e | (e, type, \tau_s) \in \mathcal{G}\}|} \tag{2}$$

As it happens in the case of frequent pattern mining [17], when extracting validating shapes, the support provides insights on how frequently a constraint is matched in the graph, i.e., the number of entities e satisfying a constraint ϕ_s. While similar to the task of itemset mining [4], the confidence can tell us how strong is the association between a node type and a specific constraint, i.e., the proportion of entities e satisfying a constraint ϕ_s among all the entities that are instances of the node type τ_s of $s \in \mathcal{S}$. For instance, the confidence for property shape headOf (Figure 1b) in our snapshot of LUBM is 10% for the Full Professor node shape and 100% for Chair, which indicates a strong association of the headOf property shape to latter and a weak association to the former.

2.3 Problem Statement

Given the need to extract shapes from a large existing graph $\mathcal{G}$ while limiting the effect of spuriousness, we formally define the problem of extracting high-quality shapes from KGs as follows:

Problem 1 (Quality Shapes Extraction). *Given an RDF graph $\mathcal{G}$, a threshold ω for support, and ε for confidence, the problem of quality shapes extraction over $\mathcal{G}$ is to find the set of shapes $\mathcal{S}$ such that for all node shapes $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ it holds that* $\text{supp}(s) > \omega$ *and for all property shapes $\phi_s{:}\langle \tau_p, T_p, C_p \rangle \in \Phi_s$,* $\text{supp}(\phi_s) > \omega$ *and* $\text{conf}(\phi_s) > \varepsilon$.

In the following, we provide both, an exact and an approximate solution to the problem of quality shape extraction.

3 QSE-EXACT

Extracting shapes $\mathcal{S}$ from an RDF graph $\mathcal{G}$ requires processing its triples and analyzing the types of nodes involved both as subjects and objects in those triples. At a high level, we need to know for each entity all its types, these will become node shapes, and then for each entity type, identify property shapes, which requires, in turn, knowing the types of the objects as well. Furthermore, we need to keep frequency counts, to know how often a specific property connects nodes of two given types compared to how many entities exist of those types. In our solution this is done in four steps: (1) entity extraction, (2) entity constraints extraction, (3) support and Confidence computation, and (4) shapes extraction. Here we first consider the case where the graph is stored as a complete dump on a single file. Later, we consider also the case for a graph stored within a triplestore [38]. Our experiments (Section 5) show that the first setup is the one that achieves the best efficiency.

QSE-Exact (file-based). One of the most common ways to store an RDF graph $\mathcal{G}$ on a file **F** is to represent it as a sequence of triples. Therefore, QSE reads **F** line by line and processes it as a stream of $\langle s, p, o \rangle$ triples. Algorithm 1 presents the four main steps of QSE to extract shapes for graph $\mathcal{G}$ stored in **F**. In the entity extraction phase, the algorithm parses each $\langle s, p, o \rangle$ triple containing a type declaration (e.g., rdf:type or wdt:P31 – this can be configured) and for each entity it stores the set of its entity types and the global count of their frequencies, i.e., the number of instances for each class (Lines 4-8) in maps Ψ_{ETD} (Entity-to-Data) and Ψ_{CEC} (Class-to-Entity-Count), respectively. In the second phase, i.e., entity constraints extraction, the algorithm performs a second pass over **F** (Lines 9-20) to collect the constraints and the meta-data required to compute support and confidence of each candidate property shape. Specifically, it parses all triples except triples containing type declarations (which can be

Algorithm 1 SHAPES EXTRACTION

Input: Graph $\mathcal{G}$ from File **F**, ω: min-support, ε: min-confidence
Output: Output: $\mathcal{S}\langle s, \tau_s, \Phi_s \rangle$

1: $E_{data} \leftarrow \{$T: $\text{SET}_{\text{Types}}$, Ψ_{ETPD} = Map$\langle$IRI, $P_{data}\rangle\}$
2: $P_{data} \leftarrow \{$T′ : $\text{SET}_{\text{ObjTypes}}$, Count :INT$\}$
3: Ψ_{ETD} = Map$\langle$IRI, $E_{data}\rangle$, Ψ_{CEC} = Map$\langle$IRI, INT$\rangle$, Ψ_{CTP} = Map$\langle$IRI, MAP$\langle$IRI, SET$\rangle\rangle$
▷ ① **Entity extraction**
4: **for** $t \in \mathcal{G} \wedge$ t.p = Type Predicate **do**
5: entity e : t.s ; entityType e_t = t.o ▷ s: subject, o: object
6: **if** $e \notin \Psi_{\text{ETD}}$ **then** Ψ_{ETD}.insert(e, ...))
7: Ψ_{ETD}.insert(e, Ψ_{ETD}.get(e).T.add(e_t)) ▷ T : entity types
8: increment entity count for current e_t in Ψ_{CEC}
▷ ② **Entity constraints extraction**
9: **for** $t \in \mathcal{G} \wedge$ t.p != Type Predicate **do**
10: $\text{Set}_{\text{ObjTypes}} \leftarrow \emptyset$, $\text{Set}_{\text{TUPLE}} \leftarrow \emptyset$ ▷ init a *type* and *property to type* tuple set
11: **if** object $t.o$ is Literal **then**
12: $\text{Set}_{\text{ObjTypes}}$.add(getLiteralType(t.o))
13: $\text{Set}_{\text{TUPLE}}$.add(new Tuple$\langle$ $t.p$, getLiteralType($t.o$)$\rangle$)
14: **else** ▷ for non-literal objects
15: **for** $\text{obj}_{\text{type}} \in \Psi_{\text{ETD}}$.get($t.o$).T **do**
16: $\text{Set}_{\text{ObjTypes}}$.add($\text{obj}_{\text{type}}$)
17: $\text{Set}_{\text{TUPLE}}$.add(new Tuple$\langle t.p, \text{obj}_{\text{type}}\rangle$)
18: addPropertyConstraints($t.s$, $\text{Set}_{\text{TUPLE}}$, Ψ_{ETD})
19: **for** IRI $\in \Psi_{\text{ETD}}$.get($t.s$.T) **do** ▷ if $t.s \in \Psi_{\text{ETD}}$
20: update Ψ_{CTP} with class IRI, $t.p$, and object types using $\text{Set}_{\text{ObjTypes}}$
▷ ③ **Support and Confidence computation**
21: Ψ_{SUPP} = MAP$\langle$TUPLE$_3$, INT$\rangle$, Ψ_{CONF} = MAP$\langle$TUPLE$_3$, INT$\rangle$, Ψ_{PTT}
22: **for** $(e, E_{data}) \in \Psi_{\text{ETD}}$ **do**
23: **for** $(T, \Psi_{\text{ETPD}}) \in E_{data}$ **do**
24: **for** $e_t \in T \wedge (p, p_o, c) \in P_{data}$ **do**
25: $\chi \leftarrow$ createTriplets($\langle \tau_e, \tau_p, \tau_{p_o}\rangle$)
26: computeSupportAndConfidence (Ψ_{SUPP}, χ, Ψ_{CEC})
27: computeMaxCardinality (Ψ_{PTT}, p , c)
▷ ④ **Shapes extraction**
28: **for** (class, MAP$\langle$Property, $\text{SET}_{\text{ObjTypes}}\rangle$) $\in \Psi_{\text{CTP}}$ **do**
29: $\Phi_s \leftarrow \emptyset$ ▷ Property shapes $\Phi_s = \{\phi_{s_1}, \phi_{s_2}, ..., \phi_{s_n}\}$ where ϕ_s:$\langle \tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle$
30: s = class.buildShapeName() , τ_s = class
31: **for** $(p, \text{SET}_{\text{ObjTypes}}) \in$ MAP$\langle$Property, SET$\rangle$ **do**
32: $\phi_s.\tau_{\text{p}} = p$
33: $p.\omega = \Psi_{\text{Supp}}$.get($p$, $\text{SET}_{\text{ObjTypes}}$) ▷ Support of property p
34: $p.\varepsilon = \Psi_{\text{Conf}}$.get($p$, $\text{SET}_{\text{ObjTypes}}$) ▷ Confidence of property p
35: **if** $p.\omega > \omega \wedge p.\varepsilon > \varepsilon$ **then**
36: **if** p is LITERAL **then** *for each* (objType $\in \text{SET}_{\text{ObjTypes}}$)
37: **if** $|\text{SET}_{\text{ObjTypes}}| > 1$ **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
38: buildConstraints(sh:nodeKind, sh:datatype, sh:maxCount, Ψ_{PTT})
39: **else** *for each* (objType $\in \text{SET}_{\text{ObjTypes}}$)
40: **if** $|\text{SET}_{\text{ObjTypes}}| > 1$ **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
41: buildConstraints(sh:nodeKind, sh:class, sh:maxCount, Ψ_{PTT})
42: $\phi_s.\text{C}_{\text{p}}.add$(sh:minCount : 1) ▷ $if p.\varepsilon > \varepsilon'$
43: $\Phi_s.add(\phi_s)$
44: $\mathcal{S}.add(s, \tau_s, \Phi_s)$ ▷ if s.$\omega > \omega \wedge \phi_s!\emptyset$

skipped now) to obtain for each predicate the subject and object types from the map Ψ_{ETD} that was populated in the previous step. The type of a literal object is inferred from the value and for non-literal object is obtained from Ψ_{ETD} map (Lines 11-17). Then, the Entity-to-Property-Data map Ψ_{ETPD} is updated to add the candidate property constraints associated with each subject entity (Line 18).

In the third phase, i.e., for support and confidence computation, the constraints' information stored in maps (Ψ_{ETD}, Ψ_{CEC}) is used to compute support and confidence for specific constraints. The algorithm iterates over the map Ψ_{ETD} to get the inner map Ψ_{ETPD} mapping entities to candidate property shapes ϕ_s:$\langle \tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle \in \Phi_s$, and retrieves the type of each entity using types information stored in Ψ_{ETD} to build triplets of the form $\langle \tau_e, \tau_p, \tau_{p_o}\rangle$ and compute their support and confidence (Line 26). The value of support and confidence for each distinct triplet is incremented in each iteration and stored in Ψ_{SUPP} and Ψ_{CONF} maps. Additionally, a map Ψ_{PTT} (Property to Types) is populated with distinct properties' frequencies and their object types in order to, later on, establish the corresponding min/max cardinality constraints (Line 27).

Finally, in the shapes extraction phase, the algorithm iterates over the values of the Ψ_{CTP} map and defines the *shape name* of s, the *shape's target definition* τ_s, and the set of *shape constraints* ϕ_s for each candidate class (Lines 28-30). The set of property shapes P for a given *Node Shape* are then extracted from the map MAP$\langle$PROPERTY, SET$\rangle$ (Lines 31-43), The C_{p} constraint can possibly have three types of values: sh:Literal, sh:IRI, and sh:BlankNode. In case of literal types, the literal object types such as xsd:string, xsd:integer, or xsd:date are used. However, in the case of non-literal object types, the constraint sh:class is used to declare the type of object to define the type of value for the candidate property. It is possible to have more than one value for the sh:class and sh:datatype constraints of a candidate property shape, e.g., to state that a property can accept both integers and floats as values, in such cases, we use sh:or constraint to encapsulate multiple values. A more detailed set of algorithms for each phase (including additional method definitions) is available in the extended version of the paper[1].

QSE-Exact (query-based). To support shapes extraction from a triplestore, we propose a query-based variant of QSE that uses a set of SPARQL queries [34] to extract all the necessary information that we collect across the four phases. In practice, we pose queries to extract all the distinct classes C, then, for each class $c \in C$, its properties $p \in \mathcal{P}$ along with object types are extracted as triplets, and support is computed for each triplet by a count query. This method is based on the standard procedure implemented also in other existing, query-based, tools [10, 18].

Shapes Pruning. QSE uses the values of support and confidence (Section 2.2) to only produce shapes satisfying the minimum threshold provided by the user. To output only meaningful shapes, QSE assures that all Property Shapes (PS) are associated with at least one Property Shape Constraint (PSc), i.e., at least a type constraint (see Definition 2.2), and that all Node Shapes (NS) are associated with at least one property shape. Hence, in fourth phase, QSE omits to create shapes not satisfying the threshold for confidence ε and support ω specified by the user (Lines 33, 34, and 42).

Cardinality Constraints. QSE supports assigning cardinality constraints (sh:minCount and sh:maxCount) to C_{p} to each property shape constraint ϕ_s:$\langle \tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle$. Following the open-world assumption, all shape constraints are initially assigned a minimum cardinality of 0, making them optional. However, there are cases where we can infer that some properties are mandatory (i.e., should be assigned a min count of 1), and some other properties can appear at most once for each entity (i.e., should be assigned both a min and a max count equal to 1). Trivially one can assign minimum cardinality 1 to property shapes having confidence 100%, i.e., for those cases in which all entities have that property. In case of incomplete KGs, QSE allows users to provide a different confidence threshold value for adding the min cardinality constraints. To achieve this, we extend the fourth phase and add a min cardinality constraint in property shapes on line 42 based on the min-confidence provided by the user. For instance, a user can instruct the algorithm to add

[1] https://github.com/dkw-aau/qse/blob/main/qse-extended.pdf

Algorithm 2 QSE-Approximate Reservoir Sampling

Input: Graph $\mathcal{G}$ from File **F**, maximum entity threshold τ_{max}, Sampling$_{\%}$
Output: Ψ_{ETD} , Ψ_{CEC}
1: init maps Ψ_{ETD} , Ψ_{SEPC}, Ψ_{RCPC} , Ψ_{CEC}, Ψ_{PC}
2: τ_{min} = 1 (minimum entity threshold) ; lineCounter = 0
3: **for** t $\in \mathcal{G}$ **do** ▷ parse s,p,o of the triple t
4: **if** t.p = Type Predicate **then**
5: entity e : t.s ; entityType e_t = t.o ▷ s: subject, o: object
6: Ψ_{SEPC}.putIfAbsent(e_t, [ ]) ▷ if $e_t \notin \Psi_{\text{SEPC}}$
7: Ψ_{RCPC}.putIfAbsent(e_t, τ_{min}) ▷ if $e_t \notin \Psi_{\text{RCPC}}$
8: **if** | Ψ_{SEPC}.get (e_t) | < Ψ_{RCPC}.get (e_t) **then** ▷ Add entity e in reservoir
9: **if** Ψ_{ETD}.get(e).T is $\emptyset$ **then** Ψ_{ETD}.insert(e, ...)) ▷ T : entity types
10: Ψ_{ETD}.insert(e, Ψ_{ETD}.get(e).T.add(t.o))
11: Ψ_{SEPC}.get(e_t).insert(e)
12: **else** ▷ Replace random entity in reservoir with current entity e
13: r = generateRandomNumber(0, lineCounter)
14: **if** r < |Ψ_{SEPC}.get(e_t) | **then**
15: $\overleftarrow{n}, \widehat{n}, \vec{n}$ = Ψ_{SEPC}.get(e_t).nodeAtIndex($r-1, r, r+1$))
16: n = getNodeWithMinimumScope($\overleftarrow{n}, \widehat{n}, \vec{n}$)
17: replace node at index n with current e & e_t in Ψ_{ETD}
18: Ψ_{SEPC}.get(e_t).add (e)
19: increment entity count for current e_t in Ψ_{CEC}
20: ▷ Resize reservoir
21: ratio = ($\Psi_{\text{SEPC}}.get(e_t).size()/\Psi_{\text{CEC}}.get(e_t)) \times 100$
22: capacity = Sampling$_{\%}$ $\times$ Ψ_{SEPC}.get(e_t).size()
23: **if** capacity < $\tau_{max} \wedge$ ratio $\leq$ Sampling$_{\%}$ **then** Ψ_{RCPC}.insert(e_t , capacity)
24: **else**$\rightarrow$ increment property count for current $t.p$ in Ψ_{PC}
25: lineCounter + +

a min cardinality constraint equal to 1 to all the property shapes that appear in at least 90% of the entities (i.e., with confidence 90%). QSE also keeps track of properties having maximum cardinality equal to 1 in a second phase and assigns sh:maxCount = 1 to those property shapes in the fourth phase while constructing shapes.

Complexity Analysis. The time complexity of QSE-Exact (Algorithm 1) is $O(2{\cdot}|F|+|E|{\cdot}|\Phi_s|+|S|{\cdot}|\Phi_s|)$. Where $2*|F|$ refers to the first and second phase having to parse all the triples twice, E is the set of entities (i.e., the set of distinct IRIs that appear as a subject for some triple), S is the set of Node Shapes, and lastly, Φ_s represents a set of all property shape constraints, i.e., $\Phi_s = \{\phi_1, \phi_2, ..., \phi_n\}$. Therefore, our algorithm scales linearly in the number of edges and nodes in the graph and in the size of the final set of shapes.

4 QSE-APPROXIMATE

QSE-Exact keeps type and property information for each entity in memory while extracting shapes. As a result, its memory requirements are prohibitively large when dealing with KGs containing hundreds of millions of nodes. To overcome this issue, we propose QSE-Approximate, an approach to enable shape extraction from very large KGs with low space complexity. Our goal is to *extract shapes by using only the resource available to a commodity machine.* QSE-Approximate is based on a multi-tiered and dynamic reservoir-sampling algorithm that we introduce. Thus, we maintain as many reservoirs as types in the graph, and we dynamically resize each reservoir as new triples are parsed. Moreover, replacement of nodes in the reservoir is performed based on the number of types of nodes in the reservoir. The resulting algorithm replaces the first phase of the QSE process. Once the sampling is performed, the information about the sampled entities is used in the same way as before in the remaining phases of Algorithm 1. Therefore, we aim at maintaining enough information to detect all shapes, but without the need to maintain more than a small samples of entities in memory.

Algorithm 2 receives as input a graph file **F**, sampling percentage (Sampling$_{\%}$), and maximum size of the reservoir per class (τ_{max}). After initialization, triples t of **F** are parsed (Line 3) and filtered based on whether they contain a type declaration. From these, we extract the entities to populate the Entity-to-Data map Ψ_{ETD} (Lines 4-24), while non-type triples are parsed on Line 24 to keep count of distinct properties in the Property-Count map Ψ_{PC}. QSE-Approximate maintains a reservoir for each distinct entity type e_t (e.g., maintaining a distinct reservoir of entities of type :Student and :FullProfessor) using a map of Sampled Entities per class (Ψ_{SEPC}). The Reservoir Capacity map (Ψ_{RCPC}) stores the current max capacities for the reservoir for each e_t. If e_t does not exist in Ψ_{SEPC} and Ψ_{RCPC}, i.e., if it has not a reservoir, one is created (lines 6-7). Then, e is inserted in the reservoir for e_t (Lines 8-11). If the reservoir has reached its current capacity limit, neighbor-based dynamic reservoir sampling is performed (Lines 13-18), i.e., a random number r is generated between zero and the current number of type declarations read from **F**. If r falls within the size of the reservoir, then a node in the reservoir is replaced with e. To select which node to replace, we identify as $\widehat{n}$ the target node at index r, and with $\overleftarrow{n}$ and $\vec{n}$ its neighbors at indexes $r{-}1$ and $r{+}1$ respectively. Among these, the node having minimum scope (i.e., minimum number of types that are known at this point in time) is selected to be replaced by the current e (Line 17). Additionally, the algorithm keeps track of actual Class-to-Entity-Count in Ψ_{CEC} (Line 19). Once the reservoir for e_t is updated, the sampling ratio for this type is computed, i.e., the proportion of entities kept so far with type e_t over the total number of entities of that type seen up now. Given the current sampling ratio and the target sampling ratio (Sampling$_{\%}$) provided as input, the algorithm evaluates whether to resize the reservoir for e_t, if it has not reached already the limit τ_{max} (Lines 21-23).

While performing shapes pruning using counts over sampled entities, QSE-Approximate requires to estimate the actual support $\overline{\omega}_\phi$ and confidence $\overline{\varepsilon}_\phi$ of a property shape ϕ from the current values ω and ε computed from the sampled data. Thus, it estimates with $\overline{\omega}_\phi{=}\omega_\phi/min(|P_r^*|/|P|, |T_r|/|T|)$ the effective support for a property shape ϕ, where ω_ϕ is the support computed for ϕ in the current sample, P represents all triples in $\mathcal{G}$ having property τ_p, P_r^* represents triples having property τ_p across all entities in all reservoirs, T represents all entities of type e_t in $\mathcal{G}$, and T_r represents all entities of type e_t in the reservoir. Similarly, the confidence $\overline{\varepsilon}_\phi$ of a property shape is estimated by replacing the denominator in equation 2 with $|T_r|$. We provide a detailed account of these computations in the extended version of the paper[1].

Space Analysis. The space requirement of QSE-Approximate depends on the values of the sampling target Sampling$_{\%}$, the maximum reservoir size τ_{max}, and the number of entity types |T| in the the graph. In the worst case, it requires $O(2{\cdot}|T|{\cdot}\tau_{max})$, therefore while the graph can contain hundreds of millions of entities, we can still easily estimate how many distinct types are in the graph and select τ_{max} to fit the available memory.

5 EVALUATION

In the following, we evaluate the scalability of both the exact and approximate versions of our QSE solutions and their effectiveness in tackling the problem of *spuriousness*.

Datasets. We selected the synthetic dataset LUBM-500 [15] and three real-world datasets: DBpedia [2] downloaded on 01.10.2020, YAGO-4 [44], for which we use the subset containing instances from the English Wikipedia, downloaded on 01.12.2020, and WikiData [47], in two variants, i.e., a dump from 2015 [51] (Wdt15), used in the original evaluation of SheXer [10], and the truthy dump from September 2021 (Wdt21) filtered by removing non-English strings. Table 1 provides a comparison of their contents, while scripts to obtain the datasets are available online [36].

Implementation and Setup. We have implemented QSE algorithms in JAVA-11. QSE supports both shapes extraction from RDF files (tiples format) and a triplestore. All experiments are performed on a single machine with Ubuntu 18.04, having 16 cores and 256 GB RAM. We have used GraphDB [13] 9.9.0 to experiment with the QSE-Exact (query-based) with a maximum memory usage limit set as 16 GB. Experiments to measure running time are executed three times, and the average runtime is computed. The memory limit for QSE are defined using Java `-Xmx` option. The source code of our approach is available as open-source [36] along with experimental settings and datasets. We have also published the extracted SHACL shapes of all our datasets on Zenodo [42].

Metrics. We measure the *running time* (in minutes) to extract the shapes for a given dataset (file or triplestore), maximum *memory usage* during the extraction process, and *Shape Statistics* of the output shapes. Additionally, we also measure the time required to load each dataset in a triplestore for QSE-Exact query-based.

QSE-Exact. We use the QSE-Exact approach to extract shapes from LUBM (L), DBpedia (D), YAGO-4 (Y), and WikiData (W). The statistics of the shapes extracted from these datasets using QSE-Exact (file-based) are shown in Table 2. It shows the count of Node Shapes (NS), Property Shapes (PS), and Property Shape Constraints (PSc), i.e., literal and non-literal node types constraints. We refer to these statistics as *default shape statistics*. We initially considered SheXer [10], ShapeDesigner [3], and SHACLGEN [18] as state-of-the-art approaches [37] to compare against QSE. Among these, both ShapeDesigner and SHACLGEN load the whole graph into a triplestore similar to our QSE-Exact (query-based) approach to extract shapes. Yet, their current implementations cannot handle large datasets with more than a few million triples, and do not manage to extract shapes of datasets having more than some hundreds of classes. In our experiments, either they crashed because they tried to load the graph into an in-memory triplestore, or required multiple hours to generate shapes for datasets such as YAGO-4 (with 8,897 classes). Therefore, in the following, we focus our comparison on SheXer, which supports both file-based and query-based methods. Nonetheless, the focus of our work is on the scalable file-based approach, hence, in the following, we only briefly report on the query-based performances of our system as a reference point and leave query-based optimization to future work. Figure 2 shows the running time and Table 3 shows the memory consumption to extract shapes for all datasets using QSE-Approximate (file-based), QSE-Exact (file-based and query-based) approach, and SheXer (file-based). SheXer (file-based), with pruning disabled, required 26, 58, 117,195 minutes and 18, 33, 59, 24 GB RAM for shape extraction from DBpedia, LUBM, YAGO-4, and Wdt15 respectively, while it went out of memory for Wdt21. Instead, QSE-Exact (file-based) required 3, 8, 23, 16, and 154 minutes for shape extraction from DBpedia, LUBM, YAGO-4, Wdt15, and Wdt21 respectively. It required 16 GB RAM for all datasets except for Wdt15 and Wdt21, which required 50 and 235 GB of RAM due to the high number of entities. On the other hand, QSE-Exact (query-based) required 34, 47, 148, 72 minutes and 16 GB RAM for shape extraction from DBpedia, LUBM, YAGO-4, and Wdt15 respectively. Both SheXer and QSE-Exact (query-based) reached the time-out for Wdt21. Moreover, it took 17, 23, 78, and 71 minutes to load DBpedia, LUBM, YAGO-4, and Wdt15, respectively, into the GraphDB triplestore in our setup. Therefore, QSE-Exact (file-based) is the most efficient approach with zero overhead or requirement to load the graph in a triplestore as compared to QSE-Exact (query-based) and SheXer (file-based).

Taming spuriousness. To deal with the issue of spuriousness, we analyze the shapes extracted and kept after pruning. QSE performs *support-based shapes extraction* by producing only the shapes that have support and confidence greater than or equal to a threshold specified by the user. For instance, given a minimum support

Table 1: Size and characteristics of the datasets

	DBpedia	LUBM	YAGO-4	Wdt15	Wdt21
# of triples	52 M	91 M	210 M	290 M	1.926 B
# of objects	19 M	12 M	126 M	64 M	617 M
# of subjects	15 M	10 M	5 M	40 M	196 M
# of literals	28 M	5.5 M	111 M	40 M	904 M
# of instances	5 M	1 M	17 M	3 M	91 M
# of classes	427	22	8,902	13,227	82,693
# of properties	1,323	20	153	4,906	9,017
Size in GBs	6.6	15.66	28.59	42	234

Table 2: Shapes Statistics using QSE-Exact.

	NS	PS	Non-Literal PSc	Literal PSc
	COUNT	COUNT/AVG	COUNT/AVG	COUNT/AVG
LUBM	23	164 / 7.1	323 / 3.0	57 /1.0
DBpedia	426	11,916 / 27.9	38,454 / 6.9	5,335 /1.0
YAGO-4	8,897	76,765 / 8.6	315,413 / 14.5	50,708 / 1.0
Wdt15	13,227	202,085 / 15.2	114,890 / 3.0	106,599 / 1.0
Wdt21	82,651	2,051,538 / 24.8	3,765,953 / 5.6	1,113,856 / 1.0

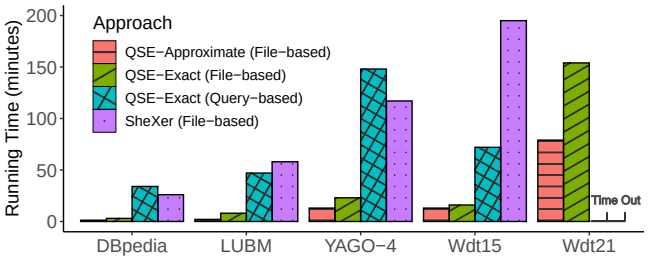

Figure 2: Running time analysis

Table 3: Memory requirement (GB) analysis of different file-based (F) and query-based (Q) shapes extraction approaches.

	D	L	Y	W15	W21
SheXer (F)	18	33	24	59	Out_M
QSE-Exact (F)	16	16	16	50	235
QSE-Exact (Q)	16	16	16	16	Out_T
QSE-Approximate (F)	**10**	**10**	**10**	**16**	**32**

threshold of 100 and minimum confidence value 25%, for every PS, QSE prunes all the PSc that do not appear with at least 100 entities or if not at least for 25% of entities for that type. We remind that the pruning of Property Shapes constraints (PSc) has a cascading effect that affects also the pruning of Property Shapes (PS), and the pruning of PS can in turn cause the pruning of Node Shapes (NS). To study the impact of various confidence and support thresholds on the number of PSc, PS, and NS, we analyze the effect of pruning by specifying various values for confidence and support.

Figure 3 shows the result of pruning PSc (3a,b), PS and NS (3c,d) for confidence > $(25, 50, 75, 90)$% and support (≥ 1, >100) on DBpedia and Wdt21, while we leave out the other datasets for brevity. In general, as expected, the results show that the higher we set the threshold for the support and confidence, the higher the percentage of PSc and PS to be pruned. Precisely, DBpedia contains $11K$ PS, $38K$ non-literal, and $5K$ literal PSc (table 2), when QSE performs pruning with confidence > 25% and support ≥ 1, it prunes out 99% of literal and non-literal PSc and PS (fig. 3a,b). Similarly for Wdt21, QSE prunes 85% non-literal and 97% literal constraints, and 66% PS for confidence >25% and support ≥1 (fig. 3b). In comparison to the default shape statistics (table 2), increasing confidence to > 50%, 75%, and 90%, pruning resulted in a drastic decrease in the number of PSc and PS. In DBpedia, the majority of non-literal PSc are pruned out, and in Wdt21, the majority of literal constraints are pruned out. Pruning of NS is lower compared to PS and PSc for all combinations of support and confidence showing that almost all types are associated at least with some very common PSc, e.g., the fact to have a name.

QSE-Approximate. QSE-Approximate approach reduces the memory requirements of the exact approach by allowing users to specify the *sampling percentage* (Sampling$_{\%}$, S% for short) and maximum limit of the *reservoir size* (τ_{max}), i.e., the maximum number of entities to be sampled per class, in order to reduce the number of entities to keep in memory. Figure 2 and Table 3 show that QSE-Approximate (file-based), with $\tau_{max} = 1000$ and S%=100%, required 1, 2, 13, 13, and 79 minutes for shapes extraction from DBpedia, LUBM, YAGO-4, Wdt15, and Wdt21 respectively. It managed to run with just 10 GB RAM on DBpedia, LUBM, and YAGO-4, and with 16 and 32 GB on Wdt15 and Wdt21, respectively. *This shows how our approach is both twice as fast and almost 1-order-of-magnitude more memory efficient.*

We further evaluate the quality of the output of QSE-Approximate using multiple combinations of values for S% and τ_{max} on Wdt21 with a fixed confidence and support threshold. Experimental results on LUBM, YAGO-4, and Wdt15 are comparable to the results presented for DBpedia and Wdt21 and are reported in the extended version of the paper[1] due to the space limitations.

We show the results in Table 4, where the values shown in columns *Real* and *Sampled* are extracted by QSE-Exact and QSE-Approximate, respectively. Here we skip listing values for Node Shapes as they are not affected by the values of S%, τ_{max}, confidence, and support. The results show that S%=10 and τ_{max} up to 200 provide a 92% precision for property shapes extracted using QSE-Approximate and pruned with support ≥ 1 and confidence >25%. This requires only 16 GB of memory and the output validating shapes are extracted in 81 minutes. In case a machine up to 24 GB memory is available, then S%=50% and τ_{max}=5K provide 96% precision with $\Delta = 93$K in 95 minutes. Similarly, on a machine having 32 GB memory, S%=100% and τ_{max}=5K provide 100% precision with $\Delta = 53$K in 98 minutes. The non-perfect precision translates into some shapes being produced despite their support and confidence being slightly lower than required. We also see that, for very small values of τ_{max} we achieve a lower recall, meaning that some shapes that should have been produced are instead wrongly pruned. We note though that min support 1 and confidence 25% are still quite low values and the shapes produced are thus more affected by spuriousness. Nonetheless, on a standard commodity machine with 32GB we see we can easily achieve perfect precision (100%) and very high recall (92%).

In Table 5 we further study the effect of pruning on shapes extracted from Wdt21 using QSE-Approximate across different support and confidence threshold. We started with pruning using confidence 25% and support 1, 10, 50, and 100. We see that with support ≥1 and confidence >25%, QSE-approximate is able to get almost all the PS extracted by QSE-Exact for Wdt21 (fig. 3d) having 89% recall and 100% precision. Additionally, upon increasing the support to 10, 50, and 100, we notice a constant recall of around 88-99% and a slight reduction in precision, i.e., 98% and 96% with decreasing relative error (i.e., Δ). Similarly, we notice the same trend with a confidence value of 75%. Therefore, while we very rarely overestimate the support and confidence of the shapes produced, we underestimate some of these values, although still in a few cases only.

Table 4: QSE-Approximate: Effect of Sampling$_{\%}$ (S%) and reservoir size (τ_{max}) on Precision (P), Recall (R), and Relative Error (Δ) with min. support 1 and confidence 25% on Wdt21

S%	τ_{max}	Property Shapes (PS)				Time (Min)	Mem (GB)
		Real	Sample	P / R	Δ		
10%	20	698,825	470,562	1.00 / 0.61	228,263	81	16
	200	698,825	497,035	0.92 / 0.65	201,790	81	16
50%	500	698,825	548,381	0.96 / 0.79	150,444	82	24
	5000	698,825	605,785	0.96 / 0.83	93,040	95	24
100%	500	698,825	617,349	1.00 / 0.88	81,476	87	32
	5000	698,825	645,810	1.00 / 0.92	53,015	98	32

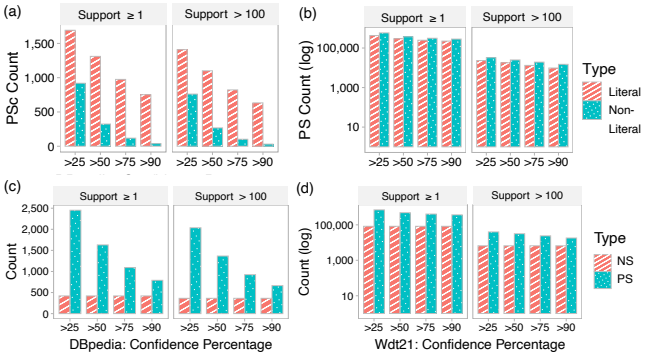

Figure 3: QSE-Exact on DBpedia and Wdt21

Table 5: Output quality of QSE-Approximate on Wdt21 with S% = 100% and $\tau_{max} = 500$ as # of real and sampled NS, PS, and corresponding Precision (P), Recall (R), and Relative Error Δ.

Conf	Supp	Node Shapes (NS)				Property Shapes (PS)			
		Real	Sample	P / R	Δ	Real	Sample	P / R	Δ
> 25 %	≥ 1	82,651	82,651	1.0 / 1.0	0	698,825	620,622	1.00 / 0.89	78,203
	10	23,640	23,640	1.0 / 1.0	0	158,283	141,040	0.99 / 0.88	17,243
	50	9,920	9,920	1.0 / 1.0	0	62,038	55,963	0.98 / 0.88	6,075
	100	6,596	6,596	1.0 / 1.0	0	39,877	36,362	0.96 / 0.88	3,515
> 75 %	≥ 1	82,651	82,651	1.0 / 1.0	0	405,344	362,717	1.00 / 0.89	42,627
	10	23,640	23,640	1.0 / 1.0	0	91,947	83,329	0.99 / 0.90	8,618
	50	9,920	9,920	1.0 / 1.0	0	37,328	34,251	0.98 / 0.90	3,077
	100	6,596	6,596	1.0 / 1.0	0	23,944	22,193	0.97 / 0.90	1,751

QSE-Approximate vs. Standard Reservoir Sampling. We studied the difference between our multi-tiered dynamic reservoir sampling in QSE-Approximate with and without the option to replace neighbors of the node. We studied the relative error for various values of sampling percentage S% (i.e., 25, 50, 75, 100) and maximum size of reservoir τ_max (i.e., 500, 1000, 1500). The results showed that QSE-Approximate without neighbor-based sampling approach extracts between 4K to 25K less property shapes, that is, by enabling neighbor-based sampling, we reduce by 50% the relative error between the approximate and the exact method.

6 RELATED WORK

KG Data Validation. Integrity constraints for KGs were initially defined with the RDF schema vocabulary [9] and then with the OWL language [25, 26, 45]. Later, the SPARQL Inferencing Notation (SPIN) [20] was proposed. SHACL [21] (a W3C standard since 2017) is known as the next generation of SPIN. Similar to SHACL, ShEX [32] is a constraint language that is built on regular bag expressions inspired by schema languages for XML. While ShEx is not a standard, it is used within the WikiData project [46]. Even though SHACL and ShEx are not completely equivalent [12], their core mechanism revolves around the same concept of enforcing for each node to satisfy specific constraints on the combination of its types and predicates [10]. In this work, we support a set of validating shapes that can be represented in both languages.

Shape Extraction. Given the abundance of large-scale KGs, various applications have been created to assist the process of extracting information about its implicit or explicit schema [19]. Among these, shapes construction or extraction approaches, i.e., to generate a set of shapes given information from an existing KG, are used in order to obtain validating schema to ensure the quality of a KG's content. We have classified existing approaches in Table 6 based on their features, i.e., support for shapes extraction from data or ontologies, support for automatic extraction of shapes, support for shapes extraction from a SPARQL triplestore, and whether they extract SHACL, ShEx, or both types of validating shapes. In our recent community survey [37] on extraction and adoption of validating shapes, we show that *there is a growing need among practitioners for techniques for efficient extraction of validating shapes from very large existing KGs.* Note that there exist approaches for schema extraction from property graphs as well [22]. Such approaches are not directly applicable to RDF KGs since their schema is more complex, moreover they focus on identifying sub-types based on node labels

Table 6: State-of-the-art to extract validating shapes [37]

Approach	*Extracted from*		*Auto-matic*	*Triple-store*	*Type*
	data	*ontology*			
Shape Induction [24]	✔	✗	✔	✔	SHACL,ShEx
SheXer [10]	✔	✗	✔	✔	SHACL,ShEx
Spahiu et al. [43]	✔	✗	✔	✔	SHACL
ShapeDesigner. [3]	✔	✗	✔	✔	SHACL,ShEx
SHACLGEN [18]	✔	✔	✔	✔	SHACL
TopBraid [35]	✔	✔	✔	✔	SHACL
Pandit et al. [30]	✗	✔	✗	✔	SHACL
Astrea [7]	✗	✔	✔	✗	SHACL
SHACLearner [28]	✔	✗	✔	✗	SHACL
Groz et al. [14]	✔	✗	✔	✗	ShEx

(which do not exist in RDF data, since types are nodes in the graph), and finally are not designed to handle the issue of spuriousness.

Rules, Patterns, and Summaries. There exist various approaches for rule discovery in graphs [23]. These systems [1, 11, 29] derive rules from large KGs using structural information by exploring the frequently occurring graph patterns. In contrast to validating shapes, rules are mainly used to derive new facts from an incomplete KG or identify specific sets of wrong connections. Frequent subgraph mining (FPM) approaches, instead, are designed to find frequently recurring structures in a large graph. In FPM, the occurrence of subgraphs (the number of times a subgraph appears) cannot be taken as the support of subgraphs since it does not satisfy the non-monotonic property [5]. The most practical measurement for measuring this support is, instead, the minimum image-based support (MNI [5]). Our proposed definition of support for shape constraints is inspired by the concept of MNI support and its use in FPM [17]. Yet, different than FPM, we do not extract patterns of arbitrary shape and size, thus we are able to provide better performance guarantees as we solve a simpler problem. Finally, our approach is also related to the techniques of graph summarization [6] and our approach can be seen as a special form of structural summarization [31].

7 CONCLUSION

Here, we propose an automatic shape extraction approach that addresses the two common limitations in other existing techniques, i.e., scalability and spuriousness. We addressed these limitations by introducing the Quality Shapes Extraction (QSE) problem. We devised an exact and approximate solution for QSE to enable the efficient extraction of shapes on commodity machines. Our method is based on the well-understood concepts of support and confidence, hence it allows a data scientist to focus on the shapes providing the highest reliability first when addressing issues of data quality. By setting even low pruning thresholds, QSE can prune up to 93% of the shapes that a trivial extraction would produce (i.e., a reduction of 2 orders of magnitude), shapes that hence have little support from the data and are thus likely spurious. Furthermore, we show that our approximate technique introduces only negligible loss in the quality and completeness of the produced shapes. In the future, we will investigate the effects of validating various large KGs against validating schemas produced by QSE and study how to extend QSE to consider rdfs:subClassOf relationship and the clustering of properties across hierarchies of types and sub-types.

REFERENCES

[1] Naser Ahmadi, Thi-Thuy-Duyen Truong, Le-Hong-Mai Dao, Stefano Ortona, and Paolo Papotti. 2020. Rulehub: A public corpus of rules for knowledge graphs. *Journal of Data and Information Quality (JDIQ)* 12, 4 (2020), 1–22.

[2] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary G. Ives. 2007. DBpedia: A Nucleus for a Web of Open Data. In *The Semantic Web, 6th International Semantic Web Conference (Lecture Notes in Computer Science)*, Vol. 4825. Springer, Busan, Korea, 722–735.

[3] Iovka Boneva, Jérémie Dusart, Daniel Fernández-Álvarez, and José Emilio Labra Gayo. 2019. Shape Designer for ShEx and SHACL constraints. In *Proceedings of the ISWC 2019 Satellite Tracks (CEUR Workshop Proceedings)*, Vol. 2456. CEUR-WS.org, Auckland, New Zealand, 269–272.

[4] Christian Borgelt. 2012. Frequent item set mining. *Wiley interdisciplinary reviews: data mining and knowledge discovery* 2, 6 (2012), 437–456.

[5] Björn Bringmann and Siegfried Nijssen. 2008. What is frequent in a single graph?. In *Proceedings of the 12th Pacific-Asia conference on Advances in knowledge discovery and data mining*. 858–863.

[6] Šejla Čebirić, François Goasdoué, Haridimos Kondylakis, Dimitris Kotzinos, Ioana Manolescu, Georgia Troullinou, and Mussab Zneika. 2019. Summarizing semantic graphs: a survey. *The VLDB journal* 28, 3 (2019), 295–327.

[7] Andrea Cimmino, Alba Fernández-Izquierdo, and Raúl García-Castro. 2020. Astrea: Automatic Generation of SHACL Shapes from Ontologies. In *ESWC (Lecture Notes in Computer Science)*, Vol. 12123. Springer, Heraklion, Crete, Greece, 497–513.

[8] WWW Consortium. 2014. RDF 1.1. https://w3.org/RDF/.

[9] WWW Consortium. 2014. RDF Schema 1.1. https://www.w3.org/TR/rdf-schema/.

[10] Daniel Fernandez-Álvarez, Jose Emilio Labra-Gayo, and Daniel Gayo-Avello. 2022. Automatic extraction of shapes using sheXer. *Knowledge-Based Systems* 238 (2022), 107975.

[11] Luis Galárraga, Christina Teflioudi, Katja Hose, and Fabian M Suchanek. 2015. Fast rule mining in ontological knowledge bases with AMIE+. *The VLDB Journal* 24, 6 (2015), 707–730.

[12] Jose Emilio Labra Gayo, Eric Prud'Hommeaux, Iovka Boneva, and Dimitris Kontokostas. 2017. Validating RDF data. *Synthesis Lectures on Semantic Web: Theory and Technology* 7, 1 (2017), 1–328.

[13] GraphDB. 2022. GraphDB. https://graphdb.ontotext.com. Accessed 20th January.

[14] Benoît Groz, Aurélien Lemay, Sławek Staworko, and Piotr Wieczorek. 2022. Inference of Shape Graphs for Graph Databases. In *25th International Conference on Database Theory (ICDT 2022)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.

[15] Y. Guo, Z. Pan, and J. Heflin. 2005. LUBM: A benchmark for OWL knowledge base systems. *Journal of Web Semantics* 3 (2005), 158–182.

[16] Claudio Gutierrez and Juan F Sequeda. 2021. Knowledge graphs. *Commun. ACM* 64, 3 (2021), 96–104.

[17] Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. 2007. Frequent pattern mining: current status and future directions. *Data mining and knowledge discovery* 15, 1 (2007), 55–86.

[18] Alexis Keely. 2022. SHACLGEN. https://pypi.org/project/shaclgen/. Accessed 20th January.

[19] Kenza Kellou-Menouer, Nikolaos Kardoulakis, Georgia Troullinou, Zoubida Kedad, Dimitris Plexousakis, and Haridimos Kondylakis. 2021. A survey on semantic schema discovery. *The VLDB Journal* (2021), 1–36.

[20] Holger Knublauch, James A Hendler, and Kingsley Idehen. 2011. SPIN-overview and motivation. *W3C Member Submission* 22 (2011), W3C.

[21] Holger Knublauch and Dimitris Kontokostas. 2017. Shapes constraint language (SHACL). *W3C Candidate Recommendation* 11, 8 (2017).

[22] Hanâ Lbath, Angela Bonifati, and Russ Harmer. 2021. Schema inference for property graphs. In *EDBT 2021-24th International Conference on Extending Database Technology*. 499–504.

[23] Michael Loster, Davide Mottin, Paolo Papotti, Jan Ehmüller, Benjamin Feldmann, and Felix Naumann. 2021. Few-shot knowledge validation using rules. In *Proceedings of the Web Conference 2021*. 3314–3324.

[24] Nandana Mihindukulasooriya, Mohammad Rifat Ahmmad Rashid, Giuseppe Rizzo, Raúl García-Castro, Óscar Corcho, and Marco Torchiano. 2018. RDF shape induction using knowledge base profiling. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC*. ACM, Pau, France, 1952–1959.

[25] Boris Motik, Ian Horrocks, and Ulrike Sattler. 2007. Adding Integrity Constraints to OWL. In *Proceedings of the OWLED 2007 Workshop on OWL: Experiences and Directions (CEUR Workshop Proceedings)*, Vol. 258. CEUR-WS.org, Innsbruck, Austria.

[26] Boris Motik, Ian Horrocks, and Ulrike Sattler. 2009. Bridging the gap between OWL and relational databases. *Journal of Web Semantics* 7, 2 (2009), 74–89.

[27] Natasha Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. 2019. Industry-scale knowledge graphs: lessons and challenges. *Commun. ACM* 62, 8 (2019), 36–43.

[28] Pouya Ghiasnezhad Omran, Kerry Taylor, Sergio José Rodríguez Méndez, and Armin Haller. 2020. Towards SHACL Learning from Knowledge Graphs. In *Proceedings of the ISWC 2020 Demos and Industry Tracks (CEUR Workshop Proceedings)*, Vol. 2721. CEUR-WS.org, Globally online, 94–99.

[29] Stefano Ortona, Venkata Vamsikrishna Meduri, and Paolo Papotti. 2018. Robust discovery of positive and negative rules in knowledge bases. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 1168–1179.

[30] Harshvardhan J. Pandit, Declan O'Sullivan, and Dave Lewis. 2018. Using Ontology Design Patterns To Define SHACL Shapes. In *Proceedings of the 9th Workshop on Ontology Design and Patterns (CEUR Workshop Proceedings)*, Vol. 2195. CEUR-WS.org, Monterey, USA, 67–71.

[31] Minh-Duc Pham, Linnea Passing, Orri Erling, and Peter Boncz. 2015. Deriving an emergent relational schema from RDF data. In *Proceedings of the 24th International Conference on World Wide Web*. 864–874.

[32] Eric Prud'hommeaux, José Emilio Labra Gayo, and Harold R. Solbrig. 2014. Shape expressions: an RDF validation and transformation language. In *Proceedings of the 10th International Conference on Semantic Systems, SEMANTiCS*. ACM, Leipzig, Germany, 32–40.

[33] E Prud'hommeaux, J. Emilio L. Gayo, and H. Solbrig. 2014. Shape expressions: an RDF validation and transformation language. In *ICSS*. 32–40.

[34] QSE-Exact 2022. SPARQL queries used for shape extraction in QSE-Exact (query-based) approach. https://github.com/kashif-rabbani/shacl/blob/main/sparql_queries.txt. Accessed 26th June.

[35] Top Quadrant. 2022. TopBraid. https://www.topquadrant.com/products/topbraid-composer/. Accessed 20th January.

[36] Kashif Rabbani. 2022. Quality Shape Extraction - resources and source code. https://github.com/dkw-aau/qse. Accessed 30th June.

[37] Kashif Rabbani, Matteo Lissandrini, and Katja Hose. 2022. SHACL and ShEx in the Wild: A Community Survey on Validating Shapes Generation and Adoption. In *Proceedings of the ACM Web Conference 2022*. ACM, Online, Lyon, France. https://www2022.thewebconf.org/PaperFiles/65.pdf

[38] Tomer Sagi, Matteo Lissandrini, Torben Bach Pedersen, and Katja Hose. 2022. A design space for RDF data representations. *VLDB J.* 31, 2 (2022), 347–373. https://doi.org/10.1007/s00778-021-00725-x

[39] Ognjen Savkovic, Evgeny Kharlamov, and Steffen Lamparter. 2019. Validation of SHACL Constraints over KGs with OWL 2 QL Ontologies via Rewriting. In *The Semantic Web - 16th International Conference, ESWC 2019, Portorož (Lecture Notes in Computer Science)*, Vol. 11503. Springer, Slovenia, 314–329.

[40] Stefan Schmid, Cory Henson, and Tuan Tran. 2019. Using Knowledge Graphs to Search an Enterprise Data Lake. In *The Semantic Web: ESWC 2019 Satellite Events - ESWC (Lecture Notes in Computer Science)*, Vol. 11762. Springer, Portorož, Slovenia, 262–266. https://doi.org/10.1007/978-3-030-32327-1_46

[41] Juan Sequeda and Ora Lassila. 2021. Designing and Building Enterprise Knowledge Graphs. *Synthesis Lectures on Data, Semantics, and Knowledge* 11, 1 (2021), 1–165.

[42] SHACL Shapes 2022. SHACL shapes for DBpedia, LUBM, YAGO-4, and WikiData published on Zenodo. https://doi.org/10.5281/zenodo.5958985. Accessed 26th June.

[43] Blerina Spahiu, Andrea Maurino, and Matteo Palmonari. 2018. Towards Improving the Quality of Knowledge Graphs with Data-driven Ontology Patterns and SHACL. In *Emerging Topics in Semantic Technologies - ISWC 2018 Satellite Events (Studies on the Semantic Web)*, Vol. 36. IOS Press, Satellite, USA, 103–117.

[44] Thomas Pellissier Tanon, Gerhard Weikum, and Fabian M. Suchanek. 2020. YAGO 4: A Reason-able Knowledge Base. In *The Semantic Web - 17th International Conference, ESWC (Lecture Notes in Computer Science)*, Vol. 12123. Springer, Heraklion, Crete, Greece, 583–596.

[45] Jiao Tao, Evren Sirin, Jie Bao, and Deborah L. McGuinness. 2010. Integrity Constraints in OWL. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI*. AAAI Press, Atlanta, Georgia, USA.

[46] Katherine Thornton, Harold Solbrig, Gregory S Stupp, Jose Emilio Labra Gayo, Daniel Mietchen, Eric Prud'Hommeaux, and Andra Waagmeester. 2019. Using shape expressions (ShEx) to share RDF data models and to guide curation with rigorous validation. In *ESWC*. Springer, Cham, 606–620.

[47] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[48] W3C. 2022. SHACL- core constraint components. https://www.w3.org/TR/shacl/#core-components. Accessed 20th January.

[49] W3C. 2022. W3C: RDF Type. http://www.w3.org/1999/02/22-rdf-syntax-ns#type. Accessed 20th January.

[50] WESO. 2022. RDFShape. http://rdfshape.weso.es. Accessed 20th January.

[51] WikiData-2015 2022. WikiData-2015. https://archive.org/details/wikidata-json-20150518. Accessed 26th June.

A APPENDIX

In this section, we present a syntactical example of SHACL shapes and the detailed algorithms of exact and approximate QSE approach.

A.1 SHACL Syntax

In section 1, we presented a figure 1 to show an example RDF graph (1a) with its validating shapes (1b). Here, we provide a syntactical example of SHACL shapes for a :Student node shape and :takesCourse property shape in Listing 1. There exist two different types of restrictions in property shapes constraints, namely: *existential restrictions* and *property type* restrictions. The former are defined by *cardinality constraints* using sh:minCount and sh:maxCount properties, while the latter define the target type for the value node of a specific property using sh:class property.

```
sh:StudentShape   a     sh:NodeShape ;
sh:targetClass    :Student
sh:property [ a  sh:PropertyShape ;
                 sh:path     :name ;
                 sh:nodeKind sh:Literal ;
                 sh:datatype xsd:String ;
                 sh:minCount 1 ;
                 sh:maxCount 1 ;   ].
sh:property [ a  sh:PropertyShape ;
                 sh:path     :takesCourse ;
                 sh:nodeKind sh:IRI ;
                 sh:minCount 1 ;
                 sh:class    :Course ;   ].
```

Listing 1: Student Node Shape having two Property Shapes

A.2 QSE-Exact

We presented a concise version of QSE-Exact (file-based) approach in section 3 where the algorithm consists of four phases of shape extraction. Here we present detailed algorithms for all four phases of shape extraction. The core steps of phase one and two are described in Algorithm 3. It starts by reading the file **F** line by line and processes it as a stream of $\langle s, p, o\rangle$ triples. In the first pass, the algorithm parses each $\langle s, p, o\rangle$ triple containing a type declaration and for each entity it stores the set of its entity types and their frequency, i.e., the number of instances for each class (Lines 6-10) in maps Ψ_{ETD} (Entity to Data) and Ψ_{CEC} (Class to Entity Count), respectively. Once the types of all entities and their counts are computed, in the second phase, the algorithm performs a second pass by streaming over **F** again (Lines 11-28) to collect the constraints and the meta-data required to compute support and confidence of each candidate shape. Specifically, it parses the triples to obtain subject and object types of all triples except for the type defined triples as they are already processed in the first pass. Thus, while processing each triple $\langle s, p, o\rangle$, the algorithm ignores triples specifying type declarations and obtains both the subject's type and the object of all the other triples. Here, the map Ψ_{ETD}, obtained in the first phase, is used to identify the types of non-literal objects, while the map Ψ_{ETPD} (Entity to Property Data) is updated to contain the candidate property constraints associated with each entity (Lines 13-21). This information about the entity types and their count is stored in maps Ψ_{ETC} and Ψ_{CEC}, respectively. Additionally, the types of objects for each particular property of a class are extracted for all classes and stored in a map Ψ_{ETP}.

Algorithm 3 EXTRACT CONSTRAINTS (Phase 1 and 2)

Input: Graph $\mathcal{G}$ from File **F**
Output: Ψ_{ETD} , Ψ_{CEC} , Ψ_{CTP}

1: EntityData ⟵ {T: $\text{SET}_{\text{Types}}$, Ψ_{ETPD} = Map⟨IRI, PropertyData⟩ }
2: PropertyData ⟵ {T′ : $\text{SET}_{\text{ObjTypes}}$, Count :INT }
3: Ψ_{ETD} = Map⟨IRI, ENTITYDATA⟩ ▹ Entity to entity's data
4: Ψ_{CEC} = Map⟨IRI, INT⟩ ▹ Class to entity count
5: Ψ_{CTP} = Map⟨IRI, MAP⟨IRI, SET⟩⟩ ▹ Class to props with object types
6: **for** $t \in \mathcal{G}$ ∧ t.p = Type Predicate **do** ▹ ① Entity extraction
7: **if** t.s ∉ Ψ_{ETD} **then**
8: Ψ_{ETD}.insert($t.s$, new EntityData (SET.init($t.o$)))
9: Ψ_{ETD}.insert($t.s$, Ψ_{ETD}.get($t.s$).T.add($t.o$))
10: Ψ_{CEC}.putIfAbsent($t.o$, SET.init(0))
11: Ψ_{CEC}.insert($t.o$, SET.add(Ψ_{CEC}.get($t.o$)+1))
12: **for** $t \in \mathcal{G}$ ∧ t.p != Type Predicate **do** ▹ ② Entity constraints extraction
13: $\lambda_{\text{TYPE}(t.o)}$ ⟵ ∅ , λ_{TUPLE} ⟵ ∅ ▹ Initialize a *type* and *property to type* tuple set
14: **if** $t.o$ is Literal **then**
15: $\lambda_{\text{TYPE}(t.o)}$.add(getType(t.o))
16: λ_{TUPLE}.add(TUPLE ⟨ t.p, getType(t.o) ⟩)
17: **else**
18: **for** IRI ∈ Ψ_{ETD}.get($t.o$) **do**
19: $\lambda_{\text{TYPE}(t.o)}$.add(IRI)
20: λ_{TUPLE}.add(TUPLE ⟨t.p, IRI⟩)
21: *updatePropertyConstraints*(t.s, λ_{TUPLE})
22: **for** IRI ∈ Ψ_{ETD}.get($t.s$.T) **do** ▹ if $t.s \in \Psi_{\text{ETD}}$
23: **if** Ψ_{CTP}.get(IRI) is ∅ **then**
24: Ψ_{CTP}.insert(IRI, ψ.init($t.p$, $\lambda_{\text{TYPE}(t.o)}$))
25: **if** Ψ_{CTP}.get(IRI).containsKey($t.p$) **then**
26: Ψ_{CTP}.get(IRI).get($t.p$).add($\lambda_{\text{TYPE}(t.o)}$)
27: **else**Ψ_{CTP}.get(IRI).insert($t.p$, $\lambda_{\text{TYPE}(t.o)}$)
28: **function** *updatePropertyConstraints (IRI, λ_{TUPLE})*
29: entityData = Ψ_{ETD}.get(IRI)
30: **for** tuple t ∈ λ_{TUPLE} **do**
31: propertyData = entityData.Ψ_{ETPD}.get($t._1$)
32: **if** propertyData is NULL **then**
33: propertyData = new PropertyData()
34: entityData.Ψ_{ETPD}.insert($t._1$, propertyData)
35: propertyData.T'.add($t._2$) ; propertyData.Count += 1
36: Ψ_{ETD}.insert(IRI, entityData)

The constraints' information extracted in the form of maps (Ψ_{ETD}, Ψ_{CEC}) in Algorithm 3 is used as input by the Algorithm 4 in the third phase to compute support and confidence for specific constraints. In this phase, the algorithm iterates over the map Ψ_{ETD} to extract the map Ψ_{ETPD} mapping entities to candidate property shapes ϕ_s:$\langle \tau_p, \mathsf{T}_p, \mathsf{C}_p\rangle \in \Phi_s$, and retrieves the type of each entity using types information stored in Ψ_{ETD} to build triplets for each distinct triplet of the form $\langle \tau_s, \tau_p, \tau_o\rangle$ (Lines 3-10). The value of support for each distinct triplet is incremented in each iteration and stored in the map Ψ_{SUPP}. Additionally, a map Ψ_{PTT} (Property to Types) is populated with properties and their object types for all the properties having max count equal to one (Lines 11-14). Once the support for each distinct triplet is computed, the value of confidence for each triplet is computed using Equation 2 and stored in the map Ψ_{CONF} (Lines 15-16).

We now use the constraints' information extracted in the first phase and the support/confidence computed in the third phase to construct the final set of shapes in phase four (presented in Algorithm 5). The algorithm iterates over the values of the Ψ_{CTP} map and defines the *shape name* of s, the *shape's target definition* τ_s, and the set of *shape constraints* ϕ_s for each candidate class (Lines 1-3). The map value MAP⟨$\text{IRI}_{\text{PROPERTY}}$, SET⟩ for each candidate class is used to extract properties and object types to define property constraints

P for each *Node Shape* (Lines 4-22). The property constraints specification for P includes sh:path, and depending on sh:nodeKind either sh:class, or sh:datatype. As a matter of fact, the sh:nodeKind constraint can possibly have three types of values: sh:Literal, sh:IRI, and sh:BlankNode. In case of Literal types, the literal object type is used, e.g., xsd:string, xsd:integer, or xsd:date. However, in the case of non-literal object types, the constraint sh:class is used to declare the type of object associated with the candidate property. It is possible to have more than one value for the sh:class and sh:datatype constraints of a candidate property shape, in such cases, we use sh:or constraint to encapsulate multiple values.

Algorithm 4 Compute Support And Confidence (Phase 3)

Input: Ψ_{ETD} , Ψ_{CEC}
Output: Ψ_{SUPP} , Ψ_{CONF} , Ψ_{PTT}
1: Ψ_{SUPP} = Map⟨Tuple$_3$, Int⟩ , Ψ_{CONF} = Map⟨Tuple$_3$, Int⟩
2: Ψ_{PTT} = Map⟨Iri, Set$_{\text{Types}}$⟩ ▹ Property to types having max count one
3: **for** ⟨Iri , E_{DATA}⟩ ∈ Ψ_{ETD} **do** ▹ Ψ_{ETD} : Map⟨Iri, EntityData⟩
4: **for** τ_s ∈ E_{Data}.get(T: Set$_{\text{Types}}$) **do** ▹ To compute support
5: PC: Set$_{Tuple2}$ = constructTupleSet(E_{Data}.get(Ψ_{ETPD}))
6: **for** $T(\tau_p, \tau_o)$ ∈ PC: **do** ▹ PC: Property Constraints Tuple$_2$ Set
7: Tuple$_3$ ← $Tuple_3\langle \tau_s, \tau_p, \tau_o \rangle$
8: **if** Ψ_{SUPP}.containsKey(Tuple$_3$) **then**
9: v = Ψ_{SUPP}.get(Tuple$_3$) , Ψ_{SUPP}.insert(Tuple$_3$, v + 1)
10: **else** Ψ_{SUPP}.insert(Tuple$_3$, 1)
11: **for** ⟨Iri , P_{DATA}⟩ ∈ E_{Data}.get(Ψ_{ETPD}) **do** ▹ To track max count cardinality
12: **if** P_{Data}.get(Count) = 1 **then**
13: Ψ_{PTT}.putIfAbsent(Iri, Set.init())
14: Ψ_{PTT}.get(Iri).insert(E_{Data}.get(T: Set$_{\text{Types}}$))
15: **for** (Tuple$_3$, $supp$) ∈ Ψ_{SUPP} **do**
16: c = Ψ_{CEC}.get(Tuple3$_1$) , $conf = \frac{supp}{c}$, Ψ_{CONF}.insert(Tuple$_3$, $conf$)

Algorithm 5 Construct SHACL Shapes (Phase 4)

Input: Ψ_{CTP} Ψ_{CONF}, Ψ_{SUPP}, Ψ_{PTT} , ω: min-support, ε: min-confidence, ε': min-confidence for assigning cardinality constraint
Output: $\mathcal{S}\langle s, \tau_s, \Phi_s \rangle$
1: **for** (Iri$_{\text{Class}}$, Map⟨Iri$_{\text{Property}}$, Set⟩) ∈ Ψ_{CTP} **do**
2: $\Phi_s \leftarrow \emptyset$ ▹ $\Phi_s = \{\phi_{s_1}, \phi_{s_2}, ..., \phi_{s_n}\}$ where ϕ_s:⟨τ_{p}, T_{p}, C_{p}⟩
3: s = Iri$_{\text{Class}}$.extractShapeName() , τ_s = Iri$_{\text{Class}}$
4: **for** (p, Set$_{\text{Object Types}}$) ∈ Map⟨Iri$_{\text{Property}}$, Set⟩ **do**
5: $\phi_s.\tau_{\text{p}} = p$
6: $p.\omega$ = Ψ_{Supp}.get(p, Set$_{\text{Object Types}}$) ▹ Support of property p
7: $p.\varepsilon$ = Ψ_{Conf}.get(p, Set$_{\text{Object Types}}$) ▹ Confidence of property p
8: **if** $p.\omega > \omega \wedge p.\varepsilon > \varepsilon$ **then**
9: **if** p is Literal **then** *for each* (objType ∈ Set$_{\text{Object Types}}$)
10: **if** |Set$_{\text{Object Types}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
11: $\phi_s.\text{T}_{\text{p}}.add$(sh:nodeKind : Literal)
12: $\phi_s.\text{T}_{\text{p}}.add$(sh:datatype :objType)
13: **if** Ψ_{PTT}.containsKey(p) ∧ Ψ_{PTT}.get(p).exists(objType) **then**
14: $\phi_s.\text{C}_{\text{p}}.add$(sh:maxCount : 1)
15: **else** *for each* (objType ∈ Set$_{\text{Object Types}}$)
16: **if** |Set$_{\text{Object Types}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
17: $\phi_s.\text{T}_{\text{p}}.add$(sh:nodeKind : IRI)
18: $\phi_s.\text{T}_{\text{p}}.add$(sh:class :objType)
19: **if** Ψ_{PTT}.containsKey(p) ∧ Ψ_{PTT}.get(p).exists(objType) **then**
20: $\phi_s.\text{C}_{\text{p}}.add$(sh:maxCount : 1)
21: $\phi_s.\text{C}_{\text{p}}.add$(sh:minCount : 1) ▹ $if p.\varepsilon > \varepsilon'$
22: $\Phi_s.add(\phi_s)$
23: $\mathcal{S}.add(s, \tau_s, \Phi_s)$ ▹ if s.$\omega > \omega \wedge \phi_s!\emptyset$

A.3 QSE-Approximate

In section 4, we presented QSE-approximate approach with an abstract level of its neighbor-based dynamic reservoir sampling (NbDRS) algorithm. Here, we present a detailed version of NbDRS algorithm. The algorithm 6 presents a detailed pseudocode for the first step where graph $\mathcal{G}$ is sampled using NbDRS. It requires graph $\mathcal{G}$ (from a file **F**), sampling percentage ($\text{Sampling}_{\text{percentage}}$), and threshold for maximum number of entities to be sampled per reservoir, i.e., τ_{max}. It starts by declaring hash maps (Lines 1-5) required for storing extracted information while parsing each $\langle s, p, o \rangle$ triple t from **F** in the loop starting from Line 7. Triples are filtered based on their type predicate (such as rdf:type and wdt:P31) to extract entities' information (Line 8-40), while non-type triples are filtered on Line 41 to keep count of distinct properties using Ψ_{PC} map.

Algorithm 6 QSE Approximate - Reservoir Sampling

Input: Graph $\mathcal{G}$ from File **F**, maximum entity threshold τ_{max}, $\text{Sampling}_{\text{percentage}}$
Output: Ψ_{ETD} , Ψ_{CEC} , Ψ_{CTP}
1: Ψ_{ETD} = Map⟨Iri, EntityData⟩ ▹ Entity to entity's data : Algorithm 3 line 1-2
2: Ψ_{SEPC} = Map⟨Iri, List<Iri>⟩ ▹ Sampled entities per class
3: Ψ_{RCPC} = Map⟨Iri, Int⟩ ▹ Reservoir capacity per class
4: Ψ_{CEC} = Map⟨Iri, Int⟩ ▹ Class to entity count
5: Ψ_{PC} = Map⟨Int, Int⟩ ▹ Property count
6: τ_{min} = 1 (minimum entity threshold) ; lineCounter = 0
7: **for** t ∈ $\mathcal{G}$ **do** ▹ t denotes triple $\langle s, p, o \rangle$
8: **if** t.p = Type Predicate **then**
9: Ψ_{SEPC}.putIfAbsent(t.o, new List<>(size: τ_{max}))
10: Ψ_{RCPC}.putIfAbsent(t.o, τ_{min})
11: **if** Ψ_{SEPC}.get (t.o).size() < Ψ_{RCPC}.get (t.o) **then** ▹ Fill the reservoir
12: **if** Ψ_{ETD}.get(t.s).T is $\emptyset$ **then**
13: Ψ_{ETD}.insert(t.s, new EntityData (Set.init(t.o)))
14: Ψ_{ETD}.insert(t.s, Ψ_{ETD}.get(t.s).T.add(t.o))
15: Ψ_{SEPC}.get(t.o).insert(t.s)
16: **else** ▹ Replace random node in the reservoir
17: r = generateRandomNumberBetween(0, lineCounter)
18: **if** r < Ψ_{SEPC}.get(t.o).size() **then**
19: $\vec{n}$ = NULL ; $\overleftarrow{n}$ = NULL ; $\vec{n}.\text{SCOPE} = \infty$; $\overleftarrow{n}.\text{SCOPE} = \infty$
20: $\widehat{n}$ = Ψ_{SEPC}.get (t.o).valueAtIndex(r))
21: **if** r ! = 0 **then** ▹ Avoid first item in reservoir
22: $\overleftarrow{n}$ = Ψ_{SEPC}.get(t.o).valueAtIndex($r - 1$))
23: **if** ∃ $\overleftarrow{n}$ ∈ Ψ_{ETD} **then** $\overleftarrow{n}.\text{SCOPE}$ = Ψ_{ETD}.get($\overleftarrow{n}$).T.size()
24: **if** r ! = Ψ_{SEPC}.get(t.o).size() −1 **then** ▹ Avoid last item in reservoir
25: $\vec{n}$ = Ψ_{SEPC}.get(t.o).valueAtIndex($r + 1$))
26: **if** ∃ $\vec{n}$ ∈ Ψ_{ETD} **then** $\vec{n}.\text{SCOPE}$ = Ψ_{ETD}.get($\vec{n}$).T.size()
27: $\widehat{n}.\text{SCOPE}$ = Ψ_{ETD}.get($\widehat{n}$).T.size()
28: n = findNodeWithMinimumScope($\overleftarrow{n}, \widehat{n}, \vec{n}$)
29: ∀ Ψ_{ETD}.T → **if** ∃ t.o ∈ Ψ_{SEPC} **then** Ψ_{SEPC}.get(t.o).removeAtIndex(n)
30: Ψ_{ETD}.remove(n) ; entityData = Ψ_{ETD}.get(t.o)
31: **if** entityData is NULL **then** entityData = new EntityData()
32: entityData.T.add (t.o) ; Ψ_{ETD}.insert(t.s, entityData)
33: Ψ_{SEPC}.get(t.o).add (t.s)
34: Ψ_{CEC}.putIfAbsent(t.o, Set.init(0))
35: Ψ_{CEC}.insert(t.o, Set.add(Ψ_{CEC}.get($t.o$)+1))
36: ▹ Resize reservoir
37: currentRatio = ($\Psi_{\text{SEPC}}.get$(t.o)$.size()$/$\Psi_{\text{CEC}}.get$(t.o)) × 100
38: newCapacity = $\text{Sampling}_{\text{percentage}}$ × Ψ_{SEPC}.get(t.o).size()
39: **if** newCapacity < τ_{max} ∧ currentRatio ≤ $\text{Sampling}_{\text{percentage}}$ **then**
40: Ψ_{RCPC}.insert(t.o , newCapacity)
41: **else** Ψ_{PC}.putIfAbsent(t.p, 0) ; Ψ_{PC}.get(t.p).incrementByOne()
42: lineCounter + +

In NbDRS approach, a reservoir is maintained for each distinct entity type (e.g., reservoir for :Student and :FullProfessor in Figure 1) using a map called Ψ_{SEPC} (Sampled Entities per class). The map Ψ_{RCPC} is designated to store reservoir's capacity for each class and allows to increase its capacity dynamically. Both these maps are initialized (Line 9-10) and if the reservoir for the current type has capacity, it is filled with its entity information using Ψ_{ETD} map (Lines 11-15); otherwise a random number r is generated between zero and the current line number of **F** (Line 17). Here we use the

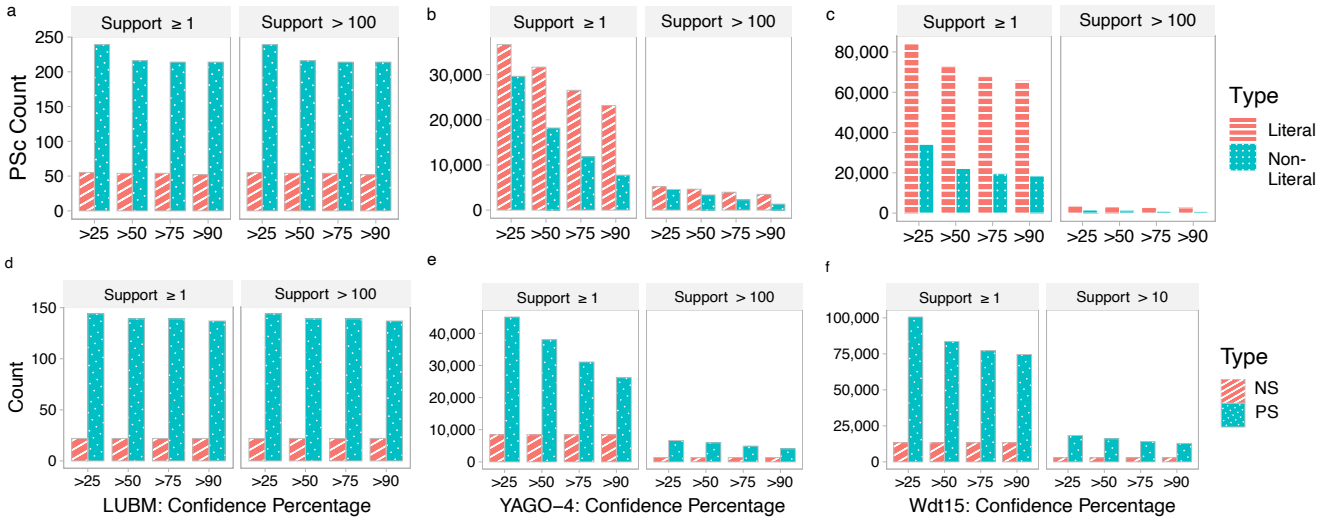

Figure 4: QSE-Exact on LUBM, YAGO-4, and Wdt15

term entity and node interchangeably. The node at index r is called focus node $\widehat{n}$, its left neighbor node $\overleftarrow{n}$ and right neighbor node $\overrightarrow{n}$ are identified and their scope is computed (Lines 19-27). The scope of a node represents the number of types of a specific entity, e.g., a person can have multiple types like Student, Teaching Assistant or a Researcher. The node with a minimum scope is marked as the node to be replaced in the reservoirs (Line 28), it is removed from the reservoirs of its type (Ψ_{SEPC}) and Ψ_{ETD} map (Lines 29-30). The new entity is added at the index of the replaced node (Line 32-33). The real entity count of each type is also computed and stored in Ψ_{CEC} (Class to Entity Count) map. This reservoir sampling approach is called dynamic as it computes the current sampling ratio (with respect to sampled entities and the total entities of the current type/class - Line 37) and new size of the reservoir using sampling percentage and count of sampled entities (Line 38). If the newly computed size is less than τ_{max} (the maximum number of entities to be sampled) and the current sampling ratio is less than the specified sampling percentage, then the reservoir capacity of the current type is increased by the newly computed reservoir's capacity (Lines 39-40). Once the sampling is performed, the sampled entities of $\mathcal{G}$ are used in the 2nd phase Algorithm 3 (From line 11).

B EXPERIMENTS

We evaluated QSE with DBpedia and Wdt21 in section 5. We could not present the results on LUBM, YAGO-4, and Wdt15 datasets in that section due to space limitations. Here we discuss the results of qualitative analysis on these datasets.

B.1 YAGO-4.

Figure 4 shows the result of PSc (4b), PS and NS (4e) pruning on YAGO-4. Our results show that applying pruning with confidence $> 25\%$ and support ≥ 1 on YAGO-4 results in the pruning of the PSc (non-literal) up to 10x compared to default shape statistics (Table 2). Increasing the support threshold to > 100 with confidence $> 25\%$ shows an interesting sudden decrease in the number of PSc of two orders of magnitude. This shows an interesting fact about entities in YAGO-4, i.e., a large portion of its taxonomy contains fewer than 100 entities. More interestingly, our results show a consistent decrease in the number of constraints when increasing the confidence threshold for all support values. Analogously to these results, Figure 4e shows the same trend for pruning of PS. In comparison to DBpedia, we noticed an interesting fact about YAGO-4 and significant reduction in the number of PSc and PS by performing shapes pruning. Analogous to these results, we notice the same decreasing trend of pruning on PS and PSc by increasing the confidence percentage to 50, 75, and 90.

B.2 Wdt15.

We performed qualitative analysis on Wdt15 using confidence values $> (25, 50, 75, 90)$ and support $\geq 1, 100$. Figure 4c shows the results of pruning performing Property Shape constraints (PSc), i.e., literal and non-literal constraints in property shapes. Given confidence 25% and support ≥ 1, QSE prunes 71% of non-literal and 22% of literal constraints. Upon increasing support to > 100, it further prunes out PSc by 2 orders of magnitude. Similarly, fig. 4f shows the results of pruning Node Shapes (NS) and Property Shapes (PS). The results show that QSE prunes 51% PS with confidence 25% and support ≥ 1 and 99% of property shapes with support > 100.

B.3 LUBM

The qualitative analysis on LUBM dataset is performed using confidence values $> (25, 50, 75, 90)$ and support $\geq 1, 100$. LUBM is a synthetic dataset, it contains 23 node shapes, 164 property shapes, 323 non-literal, and 57 literal constraints. Figure 4a shows the results of pruning performing Property Shape constraints (PSc), i.e., literal and non-literal constraints in property shapes. Similarly, fig. 4d shows the results of pruning Node Shapes (NS) and Property Shapes (PS). Due to synthetic nature of LUBM knowledge graph, pruning of PSc and PS is not significant compared to DBpedia, YAGO-4, and WikiData for defined values of support and confidence.

B.4 Cardinality Constraints

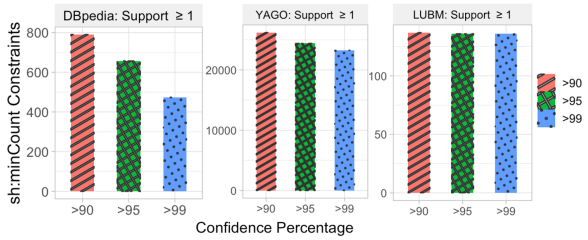

Figure 5: sh:minCount Constraint Analysis

As discussed in Section 3, QSE makes use of support and confidence computed for PSc to add a min cardinality constraint (sh:minCount) in PS. As a baseline, it assigns min cardinality to PS having a support equivalent to the number of entities of its node shape target class (Table 2). QSE also supports assigning min cardinality constraints based on the user's provided support and confidence. Figure 5 shows the number of min cardinality constraints created for PS on each dataset. Results show that the greater the value for confidence and support, the lower the number of min cardinality constraints assigned to PS.

B.5 Computational Analysis

The computational complexity of SheXer is computed as $O(2{\cdot}|T| + |E|{\cdot}\frac{|\Phi_s|^2}{|S|})$. Where $2 * |T|$ refers to the parsing of all the triples twice, E is the set of entities, S is the set of Node Shapes, and Φ_s represents a set of all property shape constraints. To compare SheXer's computational complexity against QSE (section 3), we consider Wdt21 and use statistics from Tables 1 and 2 to compute complexities. To keep it simple, we ignore the number of triples and consider number of entities $E = 91M$, node shapes $S = 82,651$, and property shapes $\Phi_s = 2M$ having literal and non literal constraints $4.8M$. The results show that the complexity of QSE-Exact is lower than SheXer by one order of magnitude. QSE-approximate allows to further reduce this complexity by choosing an appropriate values for $\text{Sampling}_{\%}$ and the τ_{max}. Although, QSE is implemented in Java and SheXer is implemented in Python, both approaches have different algorithms and complexities.